🚀 JavaScript ES6 (ECMAScript 2015

ES6 (ECMAScript 2015) introduced major enhancements to JavaScript, making it more powerful, readable, and maintainable.

1.let and const (Block Scoped Variables)

🔹 **Problem with var**

- Function scoped
- Can be redeclared
- Hoisted (causes confusion)

🔹 **ES6 Solution**

- let → block scoped, reassignable
- const → block scoped, not reassignable

✅ **Practical Demo**

```
function testVar() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 (accessible outside block)
}

function testLet() {
  if (true) {
    let y = 20;
  }
  console.log(y); // ❌ ReferenceError
}
```

🔹 **const Example**

```
const PI = 3.14;

PI = 3.1415; // ❌ Error
```

👉 Use const by default, let when reassignment is needed.

2.Arrow Functions (=>)

Shorter syntax + lexical this.

🔹 **Traditional Function**

```
function add(a, b) {
  return a + b;
}
```

🔹 **Arrow Function**

```
const add = (a, b) => a + b;
```

🔹 **Lexical this Demo**

```
function Person() {
  this.age = 0;

  setInterval(() => {
    this.age++;
    console.log(this.age);
  }, 1000);
}

new Person();
```

👉 Arrow function does NOT create its own this.

3.Template Literals (`)

Allows:

- Multiline strings
- String interpolation

✅ **Practical Demo**

```
let name = "San";
let course = "JavaScript";

console.log(`Hello ${name}, welcome to ${course}`);
```

🔹 **Multiline**

```
let message = `
This is line 1
This is line 2
`;

console.log(message);
```

4.Default Parameters

```
function greet(name = "Guest") {
   console.log(`Hello ${name}`);
}

greet();       // Hello Guest
greet("San");   // Hello San
```

5.Destructuring (Array & Object)

🔹 **Array Destructuring**

```
let numbers = [10, 20, 30];

let [a, b, c] = numbers;

console.log(a); // 10
```

🔹 **Object Destructuring**

```
let user = {
  id: 1,
  name: "San",
  role: "Trainer"
};

let { name, role } = user;

console.log(name); // San
```

6.Spread Operator (...)

Used for:

- Copying arrays
- Merging arrays
- Expanding objects

✅ **Practical Demo**

```
let arr1 = [1, 2];
let arr2 = [3, 4];
```

```
let combined = [...arr1, ...arr2];

console.log(combined); // [1,2,3,4]
```

- **Copy Array**

```
let original = [10, 20];

let copy = [...original];
```

7.**Rest Parameters**

Collects multiple arguments into an array.

```
function sum(...numbers) {

   return numbers.reduce((total, num) => total + num);

}


console.log(sum(1,2,3,4)); // 10
```

8.Enhanced Object Literals

Shorter syntax for objects.

```
let name = "San";


let user = {

   name,   // shorthand

   greet() {

      console.log("Hello!");

   }

};
```

```
user.greet();
```

9.Classes (OOP in ES6)

Cleaner syntax over prototype-based OOP.

✅ **Basic Class**

```
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    console.log(`Hello ${this.name}`);
  }
}

let p1 = new Person("San");
p1.greet();
```

🔹 **Inheritance**

```
class Employee extends Person {
  constructor(name, role) {
    super(name);
    this.role = role;
  }

  getRole() {
```

```
        console.log(this.role);
    }
}

let emp = new Employee("San", "Trainer");
emp.greet();
emp.getRole();
```

10.Modules (Import / Export)

- **Export (math.js)**

```
export const add = (a, b) => a + b;
```

- **Import (app.js)**

```
import { add } from './math.js';

console.log(add(5,3));
```

11.Promises (Asynchronous Handling)

- **Before ES6 → Callbacks**
- **ES6 → Promises**

```
let promise = new Promise((resolve, reject) => {
    let success = true;

    if (success) {
        resolve("Success!");
    } else {
        reject("Failed!");
```

```
    }
});

promise
    .then(result => console.log(result))
    .catch(error => console.log(error));
```

12.for...of Loop

Used for iterables (arrays, strings).

```
let arr = [10, 20, 30];

for (let value of arr) {
    console.log(value);
}
```

---

13. Map & Set

- **Set (Unique values)**

```
let set = new Set([1,2,2,3]);
console.log(set); // {1,2,3}
```

- **Map (Key-Value pairs)**

```
let map = new Map();

map.set("name", "San");
console.log(map.get("name"));
```

14.Iterators & Generators

🔹 **Generator Function**

```
function* generateNumbers() {
  yield 1;
  yield 2;
  yield 3;
}
let gen = generateNumbers();
console.log(gen.next().value); // 1
```

🔥 **Summary of ES6 Features**

Feature	**Purpose**
let/const	Block scope variables
Arrow functions	Short functions + lexical this
Template literals	String interpolation
Destructuring	Extract values easily
Spread/Rest	Expand & collect data
Classes	OOP
Modules	Code reuse
Promises	Async handling
Map/Set	Advanced collections
Generators	Control iteration